SENTINEL WORKERS

Autonomous Heat-Risk Prevention for Outdoor Workers

FortyGuard Hackathon 2026 - Redesigned End-to-End Product Reference

MASTER BUILD DOCUMENT

Core thesis

FortyGuard detects hyperlocal heat conditions. Sentinel Workers converts that temperature intelligence into contextual worker-risk decisions, predictive interventions, supervisor actions, and auditable operational workflows.

Project identity

Field	Decision
Project	Sentinel Workers
Primary domain	Worker safety / climate-health / operational AI
Hackathon sponsor	FortyGuard
Pilot environment	Phoenix, Arizona construction sites
Product scope	Heat-risk prevention, not merely heat visualization
Primary user	Safety supervisor / site operations lead
Secondary user	Outdoor worker
Primary differentiator	Closed-loop autonomous intervention using hyperlocal temperature intelligence + worker context
Submission deadline in supplied reference	August 30, 2026

Document purpose

This document replaces the earlier implementation reference with a safer, more defensible build specification. It consolidates the product thesis, system architecture, agent behavior, FortyGuard integration model, data contracts, risk logic, UI, demo, test plan, security, claims policy, roadmap, and submission strategy into one source of truth.

Important source rule

The original supplied reference contained implementation assumptions and numerical claims that were not all independently supported. This redesign preserves the useful architecture while separating externally documented facts, measured system behavior, simulation outputs, targets, and future capabilities.

CONTENTS

1. EXECUTIVE SUMMARY

Sentinel Workers is an autonomous worker-safety platform that turns hyperlocal heat intelligence into operational action. The product is designed around a simple loop: observe heat at the location of a worker or site, combine that environmental signal with worker and operational context, predict near-term risk, choose an intervention, execute or request that intervention, and retain a complete decision trail.

The product should not be pitched as “an AI heat dashboard.” Its winning position is “an autonomous safety system powered by hyperlocal temperature intelligence.” The dashboard is evidence of the system; it is not the product itself.

The core MVP is a Phoenix construction pilot with simulated worker profiles and real FortyGuard temperature intelligence. The system can demonstrate real-time or near-real-time monitoring, risk scoring, predictive escalation, worker notification, supervisor intervention, and audit logging. The architecture remains geography-agnostic, while current FortyGuard API constraints mean the competition build should stay within supported U.S. coverage unless the sponsor explicitly grants additional access.

Winning product statement

“FortyGuard tells us where and when heat becomes dangerous. Sentinel Workers decides who is at risk, what should happen next, and executes the safest available intervention - with a human override and a full audit trail.”

2. PROBLEM AND OPPORTUNITY

Outdoor workers can experience dangerous thermal exposure without a practical way to translate changing local heat conditions into worker-specific decisions. Conventional weather views provide environmental information, but a safety supervisor needs a different question answered: “Who is approaching unsafe exposure, what should I do now, and what happens if I do nothing?”

Primary pain points

- Heat exposure is spatially variable; a single city-level temperature is too coarse for site-level decisions.
- Workers differ in exposure duration, task intensity, shift timing, acclimatization, and risk profile.
- Pure threshold alerts are reactive and may trigger after risk has already accumulated.
- Supervisors need prioritization across many workers and sites, not hundreds of undifferentiated alerts.
- Critical interventions need accountability: the system should explain what happened, why it acted, and whether a human overrode it.

The design target

```
Observe -> Contextualize -> Predict -> Decide -> Act -> Verify -> Learn
```

The system therefore treats temperature intelligence as a decision substrate rather than the final product.

3. PRODUCT DEFINITION

Product name: Sentinel Workers

Tagline: Autonomous Heat-Risk Prevention for Outdoor Workers

North-star outcome: reduce the time between emerging thermal danger and an appropriate protective action.

Layer	MVP behavior	Production direction
Environment	Ingest FortyGuard heat/environment intelligence for supported U.S. locations	Multi-source fusion + site sensors where justified
Worker context	Profile, site, role, shift, exposure duration, risk factors	Verified workforce + access-controlled health/occupational data
Risk engine	Deterministic safety logic + contextual score	Calibrated probabilistic model with site-specific validation
Prediction	Forecast threshold breach / risk trajectory	Short-horizon spatiotemporal risk model
Action	SMS/demo notification + supervisor action	Multi-channel worker-safe delivery with acknowledgements
Oversight	Dashboard + override + audit log	Policy engine, approvals, incident management
Learning	Record outcomes and actions	Outcome feedback + model monitoring

Explicit non-goals for the hackathon

- Diagnosing heat illness or replacing medical professionals.
- Using private health data without a clear consent, governance, and security model.
- Claiming the system prevents deaths unless validated in a real deployment study.
- Pretending that a rule engine is a sophisticated autonomous agent simply because it is wrapped in an “agent” label.

4. WINNING DIFFERENTIATION

Common heat project	Sentinel Workers difference
Heat map	Maps heat -> maps heat and converts it into worker-level actions.
Weather alert	Alerts based on context and trajectory, not only a single threshold.
AI chatbot	AI is embedded in an operational loop, not the primary interface.
Generic worker app	Safety policy, site operations, escalation, and auditability are first-class.
Static ML model	Decision system combines deterministic safety guardrails with contextual reasoning.
Dashboard-only project	Dashboard is a control surface over autonomous operational workflows.

The strategic moat

```
Hyperlocal heat -> worker context -> exposure trajectory -> risk priority -> intervention -> feedback -> audit
```

Judge-proofing rule

Every “AI” claim must map to a concrete behavior: selecting the relevant evidence, prioritizing workers, predicting trajectory, choosing among permitted actions, adapting messaging, or escalating under uncertainty.

5. USER JOURNEYS

Journey A - Worker at rising-risk site

1. Worker is assigned to a construction site and active shift.
2. Sentinel retrieves the latest supported FortyGuard environmental/thermal information for the site or nearby grid.
3. Risk engine combines current conditions, exposure duration, work intensity, and forecast trajectory.
4. Worker transitions from GREEN to WATCH or elevated risk before a hard emergency threshold where possible.
5. System sends a targeted recommendation: hydrate, move to shade/cooling, take recovery break, or stop work depending on policy.
6. Worker acknowledgement or supervisor confirmation is recorded.
7. If risk continues or multiple workers become critical, escalation policy fires.

Journey B - Supervisor managing 500 workers

8. Supervisor sees prioritized risk queue instead of a wall of alerts.
9. Clicking a worker shows current conditions, exposure timeline, forecast, recent interventions, and explanation.
10. Supervisor can acknowledge, override, relocate, initiate break, or escalate.
11. The dashboard shows zone-level concentration and predicted hotspots.
12. Every consequential action is recorded with timestamp, actor, evidence, and outcome.

Journey C - Critical cluster

13. Several workers in a zone become high-risk simultaneously.
14. Agent detects spatial clustering and raises incident severity.
15. Supervisor receives one incident summary instead of redundant worker alerts.
16. Cooling/medical escalation is recommended according to site policy.
17. Incident remains open until risk is reduced and the appropriate acknowledgement is recorded.

6. SYSTEM ARCHITECTURE

```
FORTYGUARD TEMPERATURE / ENVIRONMENT INTELLIGENCE
          |
          v
+----------------------------+
| Ingestion + Normalization |
+-------------+-------------+
              |
              v
+----------------------------+      +-------------------+
| Worker / Site Context     |----->| Risk Engine       |
| - location                |      | - safety rules    |
| - shift/exposure          |      | - context score   |
| - role/intensity          |      | - uncertainty     |
| - risk profile (minimal)  |      +--------+---------+
+----------------------------+               |
                                             v
                                 +---------------------+
                                 | Prediction Agent    |
                                 | near-term trajectory|
                                 +----------+----------+
                                            |
                                            v
```

```
+---------------------+
| Action / Policy     |
| Safety-constrained  |
+---+------+------+-----+
    |      |      |
 Worker Supervisor Escalation
 notify   console   incident
    |      |      |
    +------+------+
           |
           v
+---------------------+
| Audit + Outcome Log |
+----------+----------+
           |
           v
+---------------------+
| Analytics / Learning|
+---------------------+
```

Core service boundaries

Service	Responsibility	Must not do
FG Adapter	Authentication, request construction, async polling, caching, normalization	Expose API key; invent unsupported endpoints
Heat Context Service	Map site coordinates to latest approved environmental/thermal values	Treat stale data as fresh
Worker Context Service	Maintain role/shift/exposure and minimal risk attributes	Store unnecessary sensitive health details
Risk Engine	Deterministic thresholds + contextual scoring	Override hard safety constraints
Prediction Service	Estimate near-term risk trajectory	Claim clinical prediction without validation
Policy Engine	Translate risk into permitted actions	Allow LLM to bypass safety policy
Action Service	Send notifications / supervisor tasks / escalation	Repeatedly spam users
Audit Service	Immutable-ish event trail for demo; durable event store for production	Log secrets or full sensitive payloads
Dashboard API	Serve aggregated state and events	Become the source of truth for decisions

7. AGENT DESIGN

Use a small number of clearly bounded agents. The system should demonstrate actual agent behavior without introducing unnecessary complexity.

Agent	Purpose	Inputs	Output	Autonomy boundary
Perception / Context Agent	Turns raw heat + site context into normalized evidence	FortyGuard results, site metadata, time	Context packet	Can normalize, cannot send alerts
Risk Assessment Agent	Ranks workers/sites by thermal risk	Context packet + safety policy	Risk state + reasons + confidence	Cannot override hard safety limits
Prediction Agent	Looks ahead at short-term trajectory	Historical state + forecast data	Risk-in-N-min estimate	Prediction informs action; it never disables emergency rules
Action Agent	Selects and executes allowed intervention	Risk state + policy + worker channel	Action plan + execution result	Actions limited by policy allowlist
Supervisor Agent	Summarizes incidents and recommends interventions	Clustered risks, actions, forecast	Supervisor brief + queue	Human retains final authority on non-emergency discretionary actions
Escalation Agent	Handles unresolved	Critical incidents +	Escalation event	Uses deterministic

	critical events	acknowledgement state		escalation rules; no free-form emergency improvisation

Agent contract

```
Agent input -> evidence set -> decision options -> selected action -> explanation -> confidence -> policy check -> execution -> outcome
```

LLM usage

- LLM is useful for incident summaries, operator explanations, message personalization, and converting structured evidence into human-readable recommendations.
- LLM is not the sole source of numerical risk thresholds or safety-critical decisions.
- Every LLM-generated action must pass through a deterministic policy gate before execution.
- For the hackathon, a lightweight model or deterministic planner may be sufficient; agentic quality is demonstrated by the decision loop, not model size.

8. RISK AND DECISION ENGINE

The risk engine is the heart of the redesigned product. It should explicitly separate hard safety rules from probabilistic or contextual scoring.

Stage 1 - Safety guardrails

```
IF data_stale -> downgrade confidence + supervisor flag
IF extreme_policy_condition -> mandatory protective action
IF worker_unacknowledged AND critical -> escalate
IF repeated_alert_with_no_outcome -> create incident
IF sensor/API failure -> fall back to last-known-safe policy + explicit uncertainty
```

Stage 2 - Contextual risk score

```
risk_score = w_env*environment + w_exp*exposure + w_task*task_intensity
           + w_forecast*future_trajectory + w_zone*cluster_density
           + w_worker*approved_risk_factors
           - w_recovery*recent_recovery
```

Do not hard-code clinical thresholds from this document. The implementation must keep safety thresholds configurable, versioned, and sourced from the chosen occupational-safety policy. The original reference used example WBGT bands; those should be treated as configurable demo policy rather than universal medical truth.

Stage 3 - Prediction

```
Input: last N observations + forecast window + exposure duration
Output:
  p_elevated_30m
  p_critical_60m
  expected_time_to_threshold
  uncertainty_band
```

Stage 4 - Action selection

Risk state	Primary action	Secondary action	Escalation
GREEN	Continue monitoring	Hydration/recovery reminder if policy requires	None
WATCH	Pre-emptive hydration / shade / break recommendation	Supervisor visibility	None
ELEVATED	Mandatory break / relocate according to site policy	Supervisor acknowledgement	Escalate if unresolved

HIGH	Stop/modify work per safety policy	Supervisor + site lead intervention	Open incident
CRITICAL	Immediate protective action per emergency policy	Escalate if acknowledgement fails	Emergency pathway

9. FORTYGUARD INTEGRATION

Current official FortyGuard API documentation describes a Temperature API framework with heatmaps, environmental parameters, heat intelligence, segmentation, and status tracking. The current Enterprise API uses API-key authentication and asynchronous activity IDs for submission endpoints. ṆciteÖturn634093search0Öturn634093search1Öturn634093search5Ọ

Important implementation correction

Do not copy the old integration verbatim

The original supplied reference used a direct GET temperature endpoint and a Bearer token. Current official documentation shows `api-key: YOUR_API_KEY` authentication and POST-based asynchronous tasks such as `/v1/heatmap` and `/v1/env_params`, followed by `/v1/status/{activity_id}`. Verify the exact hackathon key, plan, and enabled endpoints before coding against them.

Officially documented capabilities relevant to Sentinel Workers

- Heatmaps can generate spatial thermal layers from a polygon area of interest.
- Heatmap generation supports historical/current data and forecasting up to 12 hours ahead in the current documented release.
- Environmental Parameters can provide heat index, apparent temperature, wet bulb temperature, humidity, air-quality fields, and solar irradiance depending on plan/access.
- Heat Intelligence can produce multi-dimensional analysis including environmental and anthropogenic factors when available on the plan.
- API results use activity IDs and status polling for asynchronous operations.

Current documented API constraints relevant to this hackathon build

- Current release documentation states U.S.-only regional coverage for the listed plans.
- Heatmap granularity is documented at 60m, 80m, or 100m.
- Forecast heatmap input is currently supported up to 12 hours beyond present time.
- Plan-specific access matters; Heat Intelligence and some segmentation capabilities are Premium-only in the current documentation.
- Credits are charged on successful completion according to the documented billing model.

Integration adapter

```
class FortyGuardAdapter:
    submit_heatmap(aoi, datetime_spec) -> activity_id
    submit_env_params(lat, lon, datetime_spec) -> activity_id
    get_status(activity_id) -> result
    normalize(result) -> ThermalObservation
    cache(key, ttl)
    enforce_quota()
```

Normalized observation contract

```
ThermalObservation {
  location: {lat, lon, site_id},
  observed_at,
  temperature_c,
  apparent_temperature_c?,
  wet_bulb_c?,
  humidity_pct?,
  solar_irradiance?,
  source: "fortyguard",
  freshness_seconds,
  confidence,
  activity_id
}
```

10. DATA MODEL

Entity	Key fields
Worker	worker_id, site_id, role, shift_start, shift_end, task_intensity, channel, consent_flags
Site	site_id, name, lat, lon, zone_id, cooling_resources, emergency_policy_id
ThermalObservation	observation_id, site_id, timestamp, temperature, humidity, wet_bulb, solar, source, freshness
RiskState	worker_id, timestamp, score, level, confidence, reason_codes, forecast_breach_time
Action	action_id, worker_id/site_id, action_type, policy_version, issued_at, delivered_at, acknowledged_at, outcome
Incident	incident_id, zone_id, severity, opened_at, workers_affected, owner, closed_at, resolution
DecisionEvent	event_id, actor, input_refs, decision, explanation, policy_version, timestamp
AuditEvent	event_id, type, payload_hash/reference, created_at
ModelVersion	model_id, version, metrics, training_data_ref, deployed_at
APIUsage	provider, endpoint, activity_id, credits_estimate, status, timestamp

Privacy-by-design rule

For the hackathon, minimize personal data. Use synthetic worker identities. Avoid real medical diagnoses. If demonstrating personalization, use abstract risk attributes such as “risk_modifier = elevated” rather than storing detailed health records. Production deployment would require a separate privacy, consent, access-control, retention, and legal review.

11. BACKEND SERVICES

Recommended implementation stack remains compatible with the existing project direction: Node.js/Express for APIs and orchestration, Python for numerical/risk modeling, React for the dashboard, SQLite for a local demo and PostgreSQL for production.

Module	Key responsibilities
api/	REST endpoints, auth, dashboard queries
orchestrator/	Event loop, agent coordination, retries
fortyguard/	Provider adapter, async polling, caching
risk/	Safety policies, scoring, thresholds, explanations
prediction/	Short-horizon trajectory features/model

actions/	Notification adapters, acknowledgements
incidents/	Cluster detection, escalation workflows
audit/	Decision/event persistence
simulation/	Deterministic replay and demo scenarios
config/	Versioned policies, feature flags, endpoints

Recommended API surface

```
GET    /api/health
GET    /api/system/capabilities
GET    /api/sites
GET    /api/workers?site_id=...
GET    /api/risk/summary
GET    /api/risk/workers
GET    /api/incidents
GET    /api/events
POST   /api/simulation/start
POST   /api/simulation/stop
POST   /api/actions/:id/ack
POST   /api/actions/:id/override
POST   /api/incidents/:id/escalate
GET    /api/fortyguard/usage
```

12. FRONTEND / UX

Design goal: make risk, action, and uncertainty legible in under five seconds. The dashboard should feel like an operations console, not a data-science notebook.

Panel	What user needs to know
Global status	Are we safe, degrading, or in incident mode?
Priority queue	Which 5-10 workers/sites require attention now?
Thermal map	Where is risk concentrated?
Trajectory	What will likely worsen in the next 30-120 minutes?
Worker card	Why is this worker risky and what should happen?
Action stream	What the system did, whether it was delivered, acknowledged, or overridden
Incident view	Which risks are connected and who owns the response?
Audit drawer	What evidence and policy produced this action?

Worker detail screen

```
Worker 0042
Risk: HIGH (0.82)   Confidence: 0.91
Current condition: elevated thermal load
Exposure: 2h 43m
Forecast: critical threshold likely in 28 min
Reason codes: HEAT_RISE, LONG_EXPOSURE, HIGH_TASK_INTENSITY
Recommended action: 20-min shaded recovery + supervisor acknowledgement
Last action: warning delivered 6 min ago
[ACKNOWLEDGE] [OVERRIDE] [ESCALATE]
```

13. PREDICTION AND PERSONALIZATION

This section is the major upgrade over the original design. Do not wait until a hard threshold is crossed. Use trajectory, exposure accumulation, and site clustering to create a pre-emptive safety layer.

Features for the MVP

- Current temperature and derived thermal indicators available from FortyGuard.
- Recent change in heat conditions.
- Forecasted heat trend within the supported window.
- Exposure duration since shift start.
- Task intensity bucket: light / moderate / heavy.
- Recent recovery time.
- Worker/site risk modifier using synthetic profiles.
- Number of nearby workers entering elevated states.

MVP model options

Option	Use	Tradeoff
Rule + EWMA trend	Fastest reliable demo	Limited expressive power
Logistic regression	Interpretable risk probability	Needs labelled/replayed training data
Gradient boosting	Strong tabular performance	More complex explainability
Sequence model	Best long-term trajectory modeling	Overkill for hackathon unless data exists

Recommendation: start with deterministic guardrails + trend features + a simple interpretable probability model. The hackathon value is the end-to-end loop and measurable intervention quality, not model complexity.

14. SAFETY, GOVERNANCE, AND PRIVACY

The system is safety-adjacent. Design it so autonomy is constrained, reversible where possible, and auditable.

Control	Requirement
Hard safety policy	Emergency actions cannot be disabled by an LLM
Human override	Supervisors can override non-emergency recommendations with reason capture
Uncertainty	Every risk state includes confidence/data freshness
Fail-safe behavior	Missing data increases uncertainty and triggers a conservative operational policy
Rate limiting	Alert cooldowns and deduplication prevent notification storms
Auditability	Every consequential action references evidence and policy version
Privacy	Synthetic workers for demo; minimum necessary fields in production
Security	API keys in environment/secret store, never source control
Model governance	Versioned model, threshold configuration, and evaluation report

Human-in-the-loop policy

```
Emergency / legally mandated action -> deterministic auto-execution
Discretionary operational action -> recommendation + supervisor acknowledgement
Low-risk optimization -> autonomous by default, reversible
Unknown / conflicting evidence -> escalate uncertainty rather than hallucinate
```

15. FAILURE MODES AND FALLBACKS

Failure	Detection	Fallback	User-visible signal
FortyGuard unavailable	timeout/status failed	cached last-known observation + stale flag + local	Data degraded

		simulation	
API quota pressure	usage threshold	cache, reduce polling, batch heatmaps	Source throttled
Stale thermal data	freshness TTL	conservative policy / supervisor review	Data age shown
Notification failure	delivery callback timeout	secondary channel / supervisor alert	Undelivered
Worker does not acknowledge	ack deadline	escalate according to policy	No acknowledgement
Agent error	exception / invalid schema	policy engine takes over	Automation degraded
Model drift	monitoring metrics	rollback model	Model fallback active
Dashboard disconnected	WebSocket failure	reconnect + REST snapshot	Reconnecting
Simulation mismatch	deterministic replay checksum	known scenario dataset	Demo scenario active

16. DEMO DESIGN

Design the demo around one story, one worker, one site, and one critical cluster. The system can still simulate hundreds of workers in the background, but judges should never have to mentally assemble the story.

Time	Demo moment	Judge takeaway
0:00-0:30	Problem and stakes	This is real and important
0:30-1:10	Show FortyGuard data entering the system	Sponsor technology is central
1:10-2:00	Show worker context + predicted risk before breach	Not a basic threshold alert
2:00-3:15	Risk rises, action is selected, worker notification fires	Autonomous loop works
3:15-4:15	Supervisor view + explanation + override	Responsible autonomy
4:15-5:00	Critical cluster and incident creation	System scales operationally
5:00-5:45	Audit trail + evidence	Accountability
5:45-6:30	Impact metrics, clearly labelled as measured/simulated	Credibility
6:30-7:00	Vision: other worker-heavy sectors	Scalability

Recommended magic moment

```
“Nothing is critical yet.”
        ↓
“Forecast says this zone will cross the policy threshold in 28 minutes.”
        ↓
“Worker 0042 is predicted HIGH risk because exposure + task intensity + heat trajectory.”
        ↓
“Sentinel recommends a recovery break and moves Worker 0042 to the priority queue.”
        ↓
“Supervisor acknowledges. Worker receives action message. Event is logged.”
```

Demo credibility rules

- Never label simulated workers as real people.
- Never label synthetic phone numbers as real SMS delivery.
- If a notification is simulated, say “simulated delivery” on the UI or in the narration.
- Show real FortyGuard data where the API is actually available; do not imply that every 30-second value came directly from the provider if the provider refreshes or responds asynchronously.
- Keep a deterministic offline demo scenario so the project cannot fail because of network/API availability.

17. EVALUATION AND EVIDENCE

Create an evidence panel or README section that distinguishes four categories.

Label	Meaning	Example
MEASURED	Observed from the working system	median decision latency = X ms over N runs
SIMULATED	Produced by controlled demo scenario	487/500 profiles received simulated risk notifications
TARGET	Desired future performance	<2 min end-to-end operational response target
EXTERNAL	Supported by an external source	FortyGuard API capability / occupational guidance

Recommended metrics

- Decision latency: observation arrival to action decision.
- Action latency: decision to notification dispatch.
- Coverage: percentage of active worker profiles processed in each cycle.
- Alert precision in replayed scenarios.
- False escalation rate.
- Average lead time before policy threshold breach.
- Worker acknowledgement rate in simulation.
- Notification deduplication rate.
- API credits consumed per demo run.
- Percentage of decisions with complete audit evidence.

18. TESTING AND QA

Test layer	Required checks
Unit	Risk formulas, threshold transitions, policy evaluation, cooldowns
Contract	FortyGuard request/response schemas and status polling
Replay	Known thermal scenarios produce deterministic expected outcomes
Integration	FortyGuard adapter -> normalization -> risk -> action
Failure	API timeout, stale data, action delivery failure, disconnected UI
Security	Secret scanning, input validation, auth, rate limits
Performance	500-worker simulation, event throughput, dashboard rendering
Demo smoke	One-click start, scenario trigger, incident completion, reset

Acceptance criteria for hackathon MVP

18. A new synthetic site can be configured without changing core agent code.
19. FortyGuard adapter can submit/track supported operations and normalize results.
20. At least one predictive lead-time scenario demonstrates intervention before criticality.
21. A worker action can be acknowledged and appears in the audit trail.
22. A cluster of critical workers becomes a single supervisor incident.
23. All automated actions are policy-gated and explainable.
24. The system runs a complete offline replay if the provider is unavailable.
25. The project can be started from a clean machine using documented steps.

19. DEPLOYMENT

Hackathon deployment should favor reliability over infrastructure glamour.

Component	Hackathon	Production
Frontend	React / Vite or Next.js	Next.js + CDN
API	Node.js/Express	Containerized API behind gateway
Risk engine	Python service or same-process module	Dedicated stateless service
Queue	Optional lightweight worker queue	Redis/Kafka-style event bus as justified
DB	SQLite for demo replay	PostgreSQL
Cache	In-memory / Redis	Redis
Observability	Structured logs + simple metrics	OpenTelemetry + metrics/tracing
Secrets	`.env` locally, platform secret store	Managed secret manager
Deployment	Docker Compose	Container platform / Kubernetes only when justified

Repository structure

```
sentinel-workers/
  apps/
    api/
    dashboard/
    risk-service/
  packages/
    policy/
    schemas/
    simulation/
  providers/
    fortyguard/
  data/
    scenarios/
    synthetic-workers/
  docs/
    architecture/
    demo/
    evidence/
  infra/
    docker/
  tests/
  README.md
```

20. BUILD ROADMAP

Phase	Deliverables	Exit condition
P0 - Foundation	Repo, schemas, config, synthetic workers, basic UI	System boots with offline scenario
P1 - FortyGuard	Adapter, auth, heatmap/env calls, status polling, caching	Real provider result normalized
P2 - Risk	Safety policies, contextual score, explanations	Expected scenario outputs pass
P3 - Prediction	Trend + lead-time forecast	Pre-emptive scenario demonstrated
P4 - Actions	Notifications, ack, escalation, dedupe	Complete action loop works
P5 - Supervisor	Priority queue, map, incident view, audit	Operator can manage scenario
P6 - Hardening	Failure modes, tests, metrics, secret scan	Demo survives fault injection
P7 - Submission	Video, deck, README, claims audit	All artifacts consistent

Feature triage

Build now

Predictive lead time, contextual worker risk, FortyGuard-native integration, supervisor incident workflow, evidence labels, reliable offline replay.

Build later

Real wearable integration, large-scale event streaming, multilingual voice agent, adaptive work scheduling, city emergency integrations, model retraining pipelines.

Do not build for the hackathon

Complex Kubernetes deployment, huge foundation models, real medical diagnosis, unnecessary health-data pipelines, dozens of microservices without a measurable benefit.

21. SUBMISSION PACKAGE

The original reference recommends a five-slide deck and a 7-10 minute video. Keep the package tight and consistent across README, video, and deck. The core story should be the same everywhere.

Artifact	Content
GitHub	Clean code, one-command setup, architecture diagram, evidence, screenshots
Demo video	One end-to-end incident story with real provider integration + offline fallback
Deck	Problem -> FortyGuard advantage -> Sentinel loop -> Demo proof -> Impact / scale
README	Quickstart, architecture, agent contracts, API integration, limitations, evidence
Project description	One paragraph problem, one paragraph solution, measurable demo result, technology stack

Recommended five-slide deck

26. Problem: heat exposure is local, dynamic, and worker-specific.
27. Why FortyGuard: hyperlocal temperature intelligence becomes a new operational signal.
28. Sentinel loop: observe -> contextualize -> predict -> act -> verify.
29. Proof: live demo, measured/simulated metrics, audit trail.
30. Vision: construction first, then logistics/agriculture/utilities; same decision infrastructure.

22. JUDGE Q&A

Likely question	Recommended answer
Is this just threshold alerts?	No. Hard safety thresholds remain deterministic, but the main loop adds worker context, exposure accumulation, short-horizon prediction, prioritization, and incident clustering before a hard breach.
Where is the AI?	AI is used where uncertainty and context exist: risk prioritization, trajectory estimation, explanation, message personalization, and incident summarization. Safety rules remain deterministic.

Why FortyGuard?	Because the product needs spatially precise heat context. We are using the provider as the environmental intelligence layer instead of treating temperature as a generic city-wide number.
Can it work without FortyGuard?	The core architecture can use another provider or site sensor later, but the hackathon integration is designed around FortyGuard and explicitly shows that dependency.
Are workers real?	No. Demo workers are synthetic. That is intentional so no real person data is exposed.
Are you actually sending SMS?	Only if a real delivery provider is configured. Otherwise the demo explicitly marks notification delivery as simulated.
How do you avoid unsafe AI behavior?	Policy gates, deterministic emergency rules, confidence/freshness signals, human oversight for discretionary actions, and an audit trail.
What happens when the API fails?	The system marks data stale, reduces confidence, activates a conservative fallback, and can continue with deterministic replay/simulation.
How do you measure success?	Lead time before risk escalation, decision/action latency, false escalation rate, coverage, acknowledgement, and audit completeness.

23. METRICS AND CLAIMS POLICY

Critical rule

Never present simulation output as real-world impact. The project can be impressive without inflating the numbers.

Claim type	Example phrasing
Measured	"Across 100 replay runs, median decision latency was 180 ms."
Simulated	"In the deterministic demo scenario, 487/500 synthetic profiles received a simulated intervention."
Target	"Production target: intervention workflow under two minutes end-to-end."
External	"FortyGuard documents current support for X capability."
Future	"A production deployment could integrate emergency operations through a policy-controlled webhook."

Numbers from the original reference that need revalidation before reuse

- "45+ minutes" traditional SMS response time.
- "22x faster" comparison.
- "2 minutes" end-to-end response time.
- "13x more likely" construction-worker death rate claim.
- "~670 heat deaths annually in Arizona."
- "487/500 workers protected" unless explicitly identified as a synthetic simulation result.
- "5-15 critical cases per 12-hour shift" unless reproduced by the deterministic scenario and labelled simulated.

These figures may be usable after sourcing or measurement, but they should not be treated as established facts merely because they appeared in the prior project reference.

24. FUTURE PRODUCTION PLAN

Stage 1 - Pilot

- One organization, a small number of sites, synthetic or consented worker data, policy validation, supervised operation.

Stage 2 - Operational deployment

- Verified workforce identity, real notification delivery, incident management, monitoring, access control, data retention, and formal safety-policy review.

Stage 3 - Predictive scale

- Site-specific calibration, spatiotemporal models, outcome learning, resource optimization, and multi-site fleet management.

Stage 4 - Multi-sector

- Logistics, agriculture, utilities, road crews, outdoor events, and other heat-exposed workforces.

Potential future product modules

Module	Value
Shift Planner	Recommend safer work windows using forecast trajectories
Cooling Resource Optimizer	Position shade/water/cooling resources ahead of demand
Route Risk Engine	Avoid high-exposure routes for outdoor/mobile workers
Incident Copilot	Summarize context and response status for safety leaders
Policy Simulator	Test alternative break and exposure policies before deployment
Fleet Heat Digital Twin	Model multi-site heat risk and operational load

25. SOURCE NOTES AND REFERENCE LINKS

Primary project source: the original user-supplied master reference document, created August 10, 2026, with deadline August 30, 2026. It supplied the baseline Sentinel Workers concept, architecture, demo flow, and original implementation assumptions. The redesign intentionally preserves the product direction while correcting integration and evidence practices.

Official FortyGuard API documentation

https://docs-api.fortyguard.com/

https://docs-api.fortyguard.com/docs/quickstart

https://docs-api.fortyguard.com/docs/authentication

https://docs-api.fortyguard.com/docs/create-heatmap

https://docs-api.fortyguard.com/docs/environmental-parameters

https://docs-api.fortyguard.com/docs/heat-intelligence

https://docs-api.fortyguard.com/docs/limitations

https://docs-api.fortyguard.com/docs/release-notes

https://www.fortyguard.com/

Current documentation-derived facts used in this redesign

- FortyGuard documents heatmaps, environmental parameters, Heat Intelligence, segmentation, and other temperature-intelligence capabilities.
- Current Enterprise API documentation uses `api-key` header authentication and asynchronous `activity_id` status handling.
- Current release notes identify v1.0.0 as the initial public Enterprise API release on April 22, 2026.
- Current limitations documentation lists U.S.-only coverage for the current plans and plan-dependent feature access.
- Current heatmap documentation lists 60m/80m/100m granularity and forecast support up to 12 hours beyond the current time.

Source integrity note

The official FortyGuard documentation should be treated as the source of truth for API parameters, plan access, rate/credit behavior, and endpoint availability. The hackathon key or sponsor-specific environment may expose capabilities not described identically in public documentation; verify the actual event credentials and test them before finalizing the provider adapter.

FINAL BUILD CHECKLIST

- Project thesis is framed as autonomous safety infrastructure, not a heat dashboard.
- FortyGuard is visibly central to the system and the demo.
- Provider integration matches the current documented authentication and asynchronous task model.
- Hard safety constraints are deterministic and policy-controlled.
- Predictive lead time exists before critical threshold where data supports it.
- Worker context includes exposure and task intensity.
- Actions are deduplicated, acknowledged, and audited.
- Critical clusters create incidents rather than alert spam.
- Offline deterministic replay exists as a demo fallback.
- Every numerical claim is labelled measured, simulated, target, or external.
- No real personal health data is exposed in the hackathon demo.
- README, video, pitch deck, and dashboard tell the same story.
- All secrets are excluded from source control.
- Clean-machine setup is documented and tested.
- Final demo is timed and has a deterministic recovery path.

END OF MASTER REFERENCE

Version 2.0 - Redesigned for implementation, verification, and hackathon submission